

BORTOLUSSI Diego

CASSE Enzo

FISA INFO 25/28

DA COSTA Argan

VIEIRA YEPEZ Steven

26/06/2026

Rapport Projet Breezy

Table des matières

1	Objectifs et attentes du projet	3
1.1	Présentation et finalité	3
1.2	Objectifs fonctionnels	3
1.3	Objectifs techniques	3
2	Conception de l'interface	4
3	Architecture du projet	7
3.1	Rôle de la passerelle (API Gateway)	8
3.2	Les microservices	8
3.3	Technologies de stockages	8
3.4	Communications entre services	10
3.5	Déploiement	10
4	Modèle de données	11
4.1	Service Auth (PostgreSQL)	11
4.2	Service User (PostgreSQL)	12
4.3	Service Profile (MongoDB)	12
4.4	Service Post (MongoDB)	12
4.5	Principaux choix de conception	13
5	Mise en œuvre des fonctionnalités	13
5.1	Fonctionnalités principales	13
5.2	Périmètre réalisé	16
6	Organisation, méthodologie et déroulement du projet	17
6.1	Gestion des rôles	17
6.2	Hierarchie des tâches	18
6.3	Méthodologie de travail	18
6.4	Étapes du projet	20
6.5	Ressources mobilisées	20
7	Axes d'amélioration et évolutions potentielles	21
8	Conclusion	22

1 Objectifs et attentes du projet

1.1 Présentation et finalité

Breezy est un réseau social, comparable à Twitter/X, développé dans le cadre de ce projet. L'application permet à un utilisateur de s'inscrire, de publier des messages courts (280 caractères maximum), de suivre d'autres utilisateurs et d'interagir avec leurs publications par des likes et des réponses. Le périmètre du projet est défini par un cahier des charges de 23 user stories (Fx1 à Fx23), réparties en fonctionnalités principales et secondaires. Le projet impose une réalisation en architecture microservices, avec un frontend web séparé du backend.

1.2 Objectifs fonctionnels

Les fonctionnalités principales (Fx1 à Fx11) constituent le socle obligatoire. L'utilisateur doit pouvoir créer un compte avec validation des données (Fx1) et se connecter de façon sécurisée (Fx2). Il doit pouvoir publier des messages courts (Fx3), les retrouver sur sa page de profil (Fx4, Fx11) et consulter un fil d'actualité chronologique alimenté par les comptes qu'il suit (Fx5). Il doit pouvoir liker un post (Fx6), répondre à un post (Fx7) et répondre à un commentaire (Fx8), suivre ou être suivi par d'autres utilisateurs (Fx9), et disposer d'un profil affichant son nom, sa biographie et sa photo (Fx10).

Les fonctionnalités secondaires (Fx12 à Fx23) sont optionnelles et viennent enrichir l'application : ajout de tags aux messages (Fx12) et recherche par tags (Fx13), notifications pour les mentions, les likes et les nouveaux abonnés (Fx14 à Fx16), messagerie privée (Fx17), ajout d'images (Fx18) et de vidéos (Fx19), signalement de contenu (Fx20) et bannissement d'utilisateurs par un modérateur (Fx21), interface multilingue (Fx22) et thème personnalisable (Fx23).

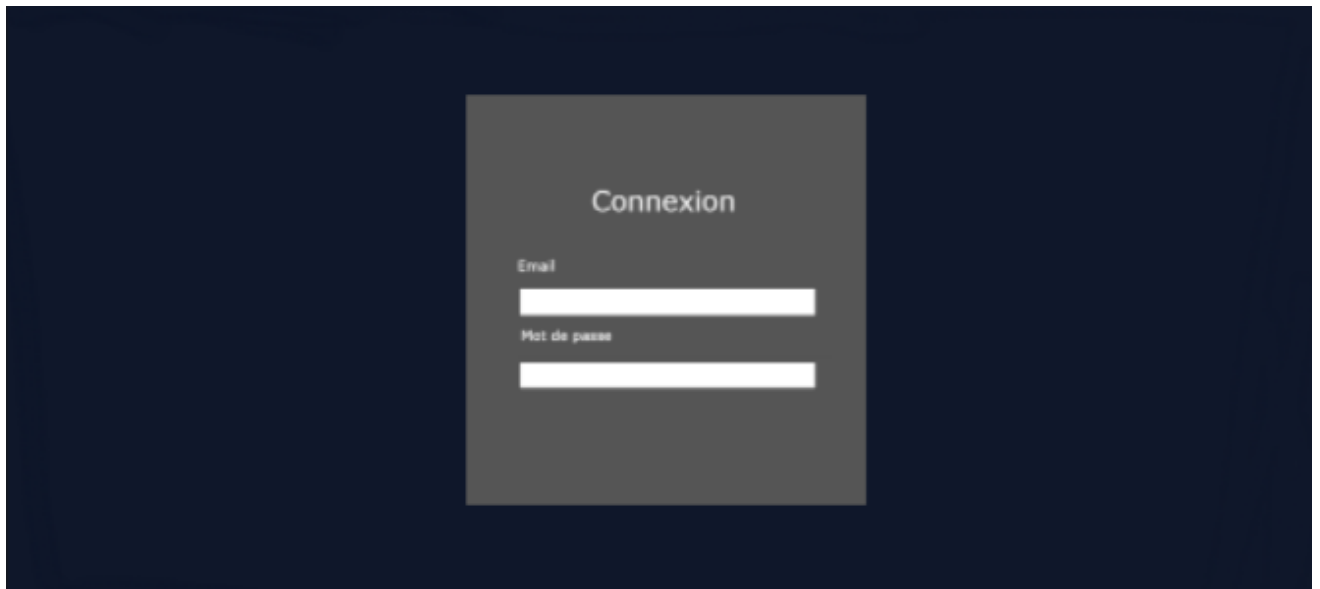
1.3 Objectifs techniques

Le projet doit être réalisé en architecture microservices : plusieurs services indépendants, chacun responsable d'un domaine (authentification, comptes utilisateurs, publications, profils et abonnements, médias) et de sa propre base de données. Les services ne sont pas exposés directement : une passerelle d'API unique sert de point d'entrée, centralise l'authentification par jeton JWT et applique un contrôle d'accès par rôles. Le projet utilise différentes technologies, avec une base relationnelle (PostgreSQL) pour les données de comptes et une base documentaire (MongoDB) pour les données sociales, ainsi qu'un

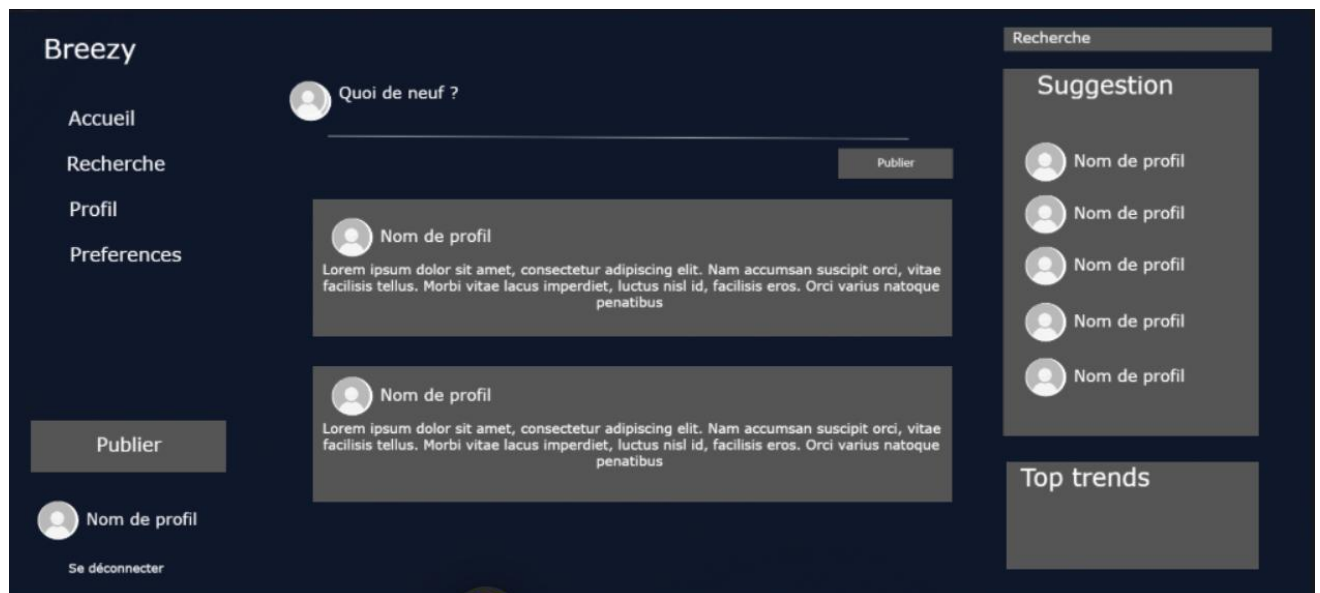
stockage objet (MinIO) pour les médias. L'ensemble doit être conteneurisé avec Docker et s'appuyer sur une intégration continue pour garantir un déploiement reproductible.

2 Conception de l'interface

Avant d'aborder l'architecture technique, cette section présente la conception des interfaces de l'application Breezy sous forme de wireframes. La conception repose sur une mise en page responsive : une organisation en trois colonnes sur écran large (navigation à gauche, contenu au centre, suggestions et tendances à droite) qui se réduit à une colonne unique accompagnée d'une barre de navigation inférieure sur mobile, conformément à l'approche mobile-first attendue.



Le premier écran constitue le point d'entrée de l'application. Il présente un formulaire comportant deux champs, Email et Mot de passe, et matérialise l'authentification sécurisée de l'utilisateur (Fx2). Cet écran sert également de porte d'accès à la création de compte (Fx1). Tant que l'utilisateur n'est pas authentifié, aucune autre fonctionnalité n'est accessible, ce qui traduit visuellement la matrice des permissions (le rôle Visiteur ne dispose que de la connexion et de l'inscription).



Une fois connecté, l'utilisateur accède à la page d'accueil, écran principal de l'application. Sa structure en trois colonnes organise l'ensemble des interactions sociales. La colonne de gauche rassemble la navigation persistante : Accueil, Recherche, Profil, Préférences, ainsi que le bouton Publier, le profil de l'utilisateur courant et l'action Se déconnecter.

La colonne centrale présente la zone de composition « Quoi de neuf ? », qui permet la publication de messages courts (Fx3), suivie du fil chronologique affichant les publications des utilisateurs suivis (Fx5). Chaque publication est présentée sous forme de carte associant l'avatar et le nom de l'auteur au contenu du message. La colonne de droite regroupe la barre de recherche (Fx13), un bloc Suggestion proposant des comptes à suivre (Fx9) et un bloc Top trends mettant en avant les tags les plus populaires (Fx12, Fx13). Cette disposition rend les actions essentielles, publier, consulter, rechercher, suivre, accessibles depuis un seul et même écran, ce qui réduit la charge de navigation et fluidifie l'expérience.

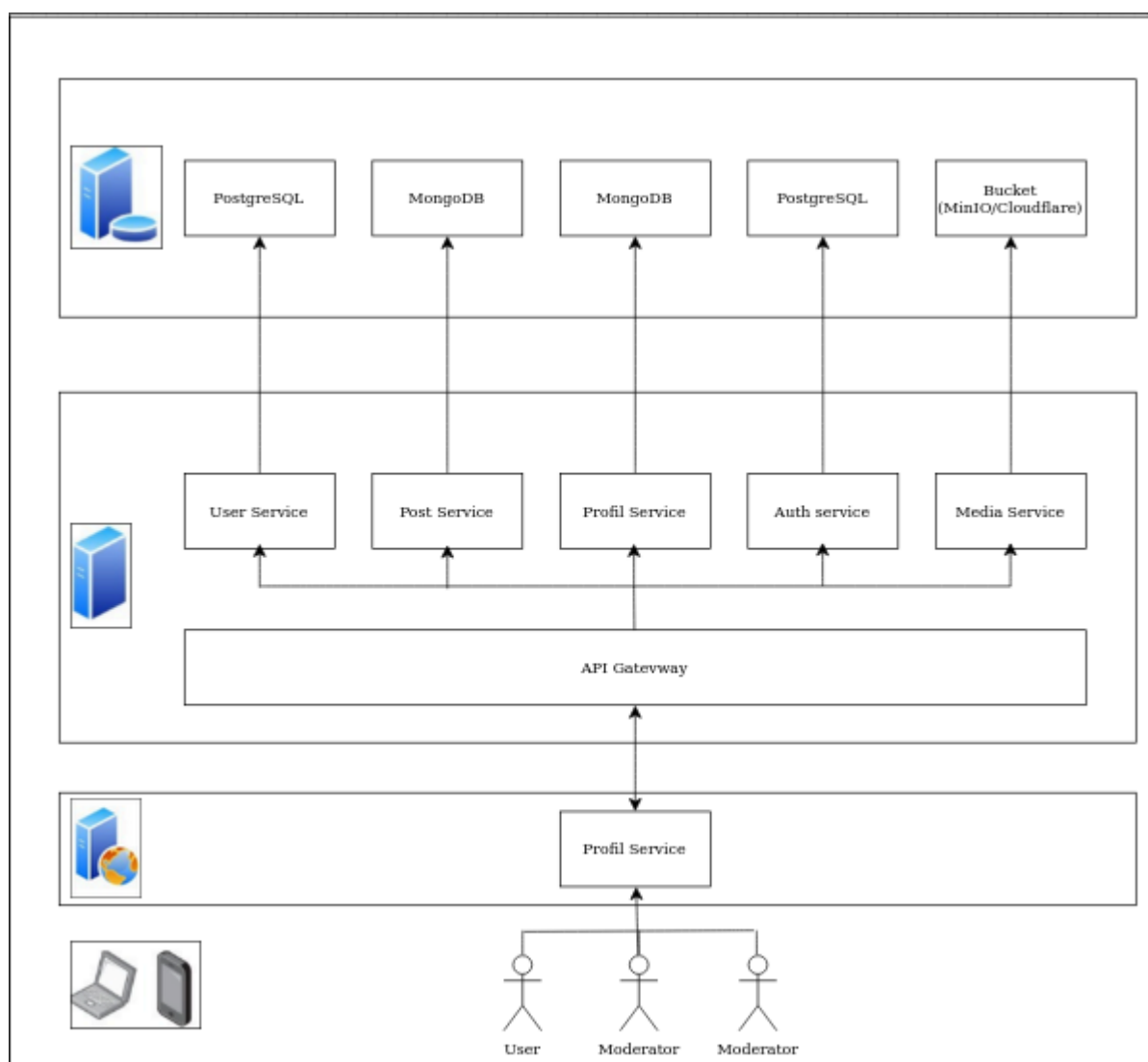


La page de profil conserve les colonnes latérales de navigation et de suggestions afin de garantir une cohérence d'ensemble entre les écrans. La zone centrale affiche les informations de base de l'utilisateur (Fx10) : avatar, nom de profil et biographie courte. Une ligne de compteurs récapitule le nombre de publications, d'abonnés et d'abonnements, donnant une lecture immédiate de l'activité et du graphe social de l'utilisateur (Fx9). En dessous, un système d'onglets, Posts, Posts et réponses, J'aime, permet de filtrer le contenu rattaché au profil : la liste des messages publiés (Fx11), les réponses apportées (Fx7, Fx8) et les publications appréciées (Fx6).

3 Architecture du projet

L'application repose sur une architecture microservices organisée autour d'une passerelle d'API unique. Le frontend (Next.js) ne communique jamais directement avec les services : il envoie toutes ses requêtes à la passerelle (port 8080), qui constitue le seul point d'entrée du backend.

Nous avons organisé en suivant le schéma initial suivant donné par l'énoncé du projet :



3.1 Rôle de la passerelle (API Gateway)

La passerelle centralise trois responsabilités. Elle authentifie chaque requête en vérifiant le jeton JWT (lu dans un cookie HttpOnly). Elle applique le contrôle d'accès par rôles (user, moderator, admin) à partir d'une table de routes associant chaque chemin à un service cible et à ses exigences de sécurité. Enfin, elle nettoie les en-têtes d'identité entrants pour empêcher toute usurpation, puis ré-injecte une identité de confiance (x-user-id, x-user-role) vers le service interne destinataire.

3.2 Les microservices

Le backend est découpé en cinq services autonomes, chacun responsable d'un domaine métier et de sa propre base :

- **auth-service** (port 5001, PostgreSQL) : Inscription, connexion, gestion des jetons d'accès et de rafraîchissement, changement de mot de passe et d'email.
- **user-service** (port 3001, PostgreSQL) : Comptes utilisateurs (nom, email, rôle, avatar, préférences) et actions de modération (suspension, bannissement, réintégration).
- **post-service** (port 3003, MongoDB) : Posts, commentaires, likes, tags et signalements de contenu.
- **profile-service** (port 3004, MongoDB) : Profils, relations d'abonnement et compteurs.
- **media-service** (port 3005, stockage objet MinIO) : Envoi et suppression des images et vidéos.

3.3 Technologies de stockages

Conformément au patron de conception architectural Database-per-service, chaque microservice possède un accès exclusif à son propre système de persistance. Cela interdit toute requête directe, jointure inter-base ou dépendance de stockage en provenance d'un service tiers, garantissant ainsi un couplage faible et une isolation totale des services.

Une infrastructure hybride SQL et NoSQL

Pour les services auth-service et user-service, nous avons sélectionné le système de gestion de base de données relationnelle PostgreSQL. Ce choix est justifié par la nécessité de manipuler des structures de données rigides, fortement typées et d'assurer des

transactions ACID strictes, indispensables à la sécurité des identifiants, à l'unicité des comptes et à la gestion des rôles (RBAC).

À l'inverse, post-service et profile-service s'appuient sur MongoDB, une base de données orientée documents (NoSQL). MongoDB stocke les informations sous forme de documents JSON/BSON flexibles, un format idéal pour modéliser des structures arborescentes ou variables, comme les fils de commentaires imbriqués, les listes de tags fluctuantes ou l'évolution rapide des profils publics et de leurs compteurs d'abonnés.

Le stockage objet avec MinIO et le concept de "Buckets"

Enfin, le media-service utilise MinIO, un serveur de stockage d'objets haute performance et entièrement compatible avec l'API standard Amazon S3. Contrairement à un système de fichiers traditionnel (qui organise les données en arborescence de dossiers) ou à une base de données classique (qui utilise des tables), le stockage objet manipule les fichiers comme des unités autonomes (des objets) associées à des métadonnées et à un identifiant unique.

Pour organiser et isoler ces objets, MinIO repose sur le concept fondamental de Buckets (ou "Seaux"). Un bucket est un conteneur logique de premier niveau dans lequel sont téléversés les fichiers. On peut le comparer à un répertoire racine ou à un disque virtuel isolé, fonctionnant avec un espace de nommage plat (sans sous-dossiers physiques réels).

Dans le cadre de Breezy, l'ensemble des médias est centralisé dans un bucket unique, breezy-media. L'isolation entre utilisateurs repose sur une organisation par préfixe : chaque fichier est rangé sous un chemin de la forme identifiant-utilisateur/UUID.extension.

Lorsqu'un utilisateur publie un média, le media-service intercepte le fichier binaire, déduit son type (image ou vidéo) à partir de son type MIME, lui attribue un identifiant unique universel (UUID) qui garantit l'unicité du nom d'objet, puis le pousse dans le bucket. Le service renvoie enfin l'URL d'accès au fichier, le bucket étant configuré en lecture seule publique pour permettre son affichage direct dans l'application.

Cette architecture apporte deux bénéfices majeurs au projet :

- Une scalabilité maximale : Les buckets s'affranchissent des limites de performance des systèmes de fichiers classiques, qui s'effondrent souvent lorsque des dizaines de milliers de fichiers s'accumulent dans un même répertoire.
- Le soulagement des bases de données principales : En stockant les lourds fichiers binaires en dehors de PostgreSQL et de MongoDB, nous évitons d'engorger ces moteurs. Les bases textuelles restent légères, ce qui préserve la rapidité des requêtes, des indexations et des sauvegardes du système.

3.4 Communications entre services

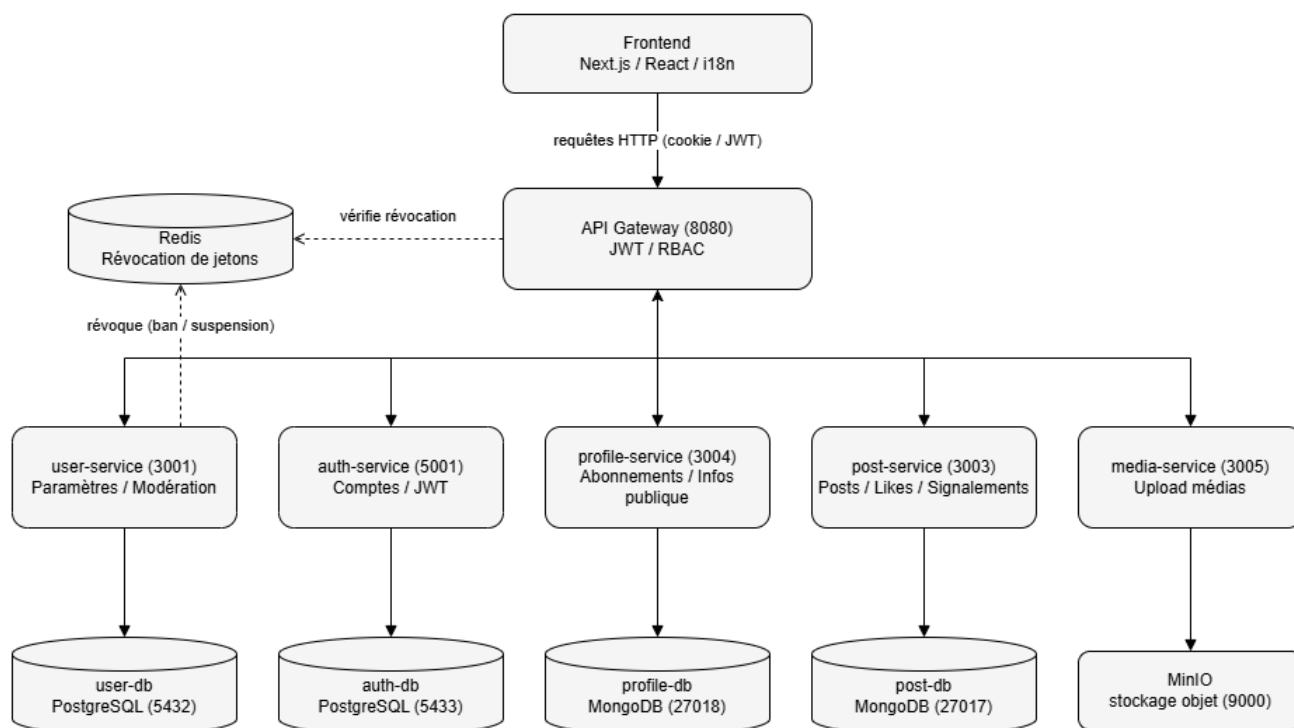
Les services ne s'appellent jamais directement : toute communication interne transite par la passerelle, qui sert aussi de routeur. Deux cas : des appels synchrones via la passerelle (inscription -> /api/users puis /api/profile / suppression d'un contenu -> /api/media), et un mécanisme indirect via Redis pour la révocation des jetons lors d'une suspension ou d'un bannissement.

3.5 Déploiement

L'ensemble (cinq services, quatre bases de données, Redis, un stockage objet MinIO, la passerelle) est conteneurisé et orchestré par Docker Compose, avec une base de données par conteneur et des volumes persistants. Le frontend Next.js est exécuté séparément et pointe vers la passerelle.

Voici la représentation visuelle de cette architecture.

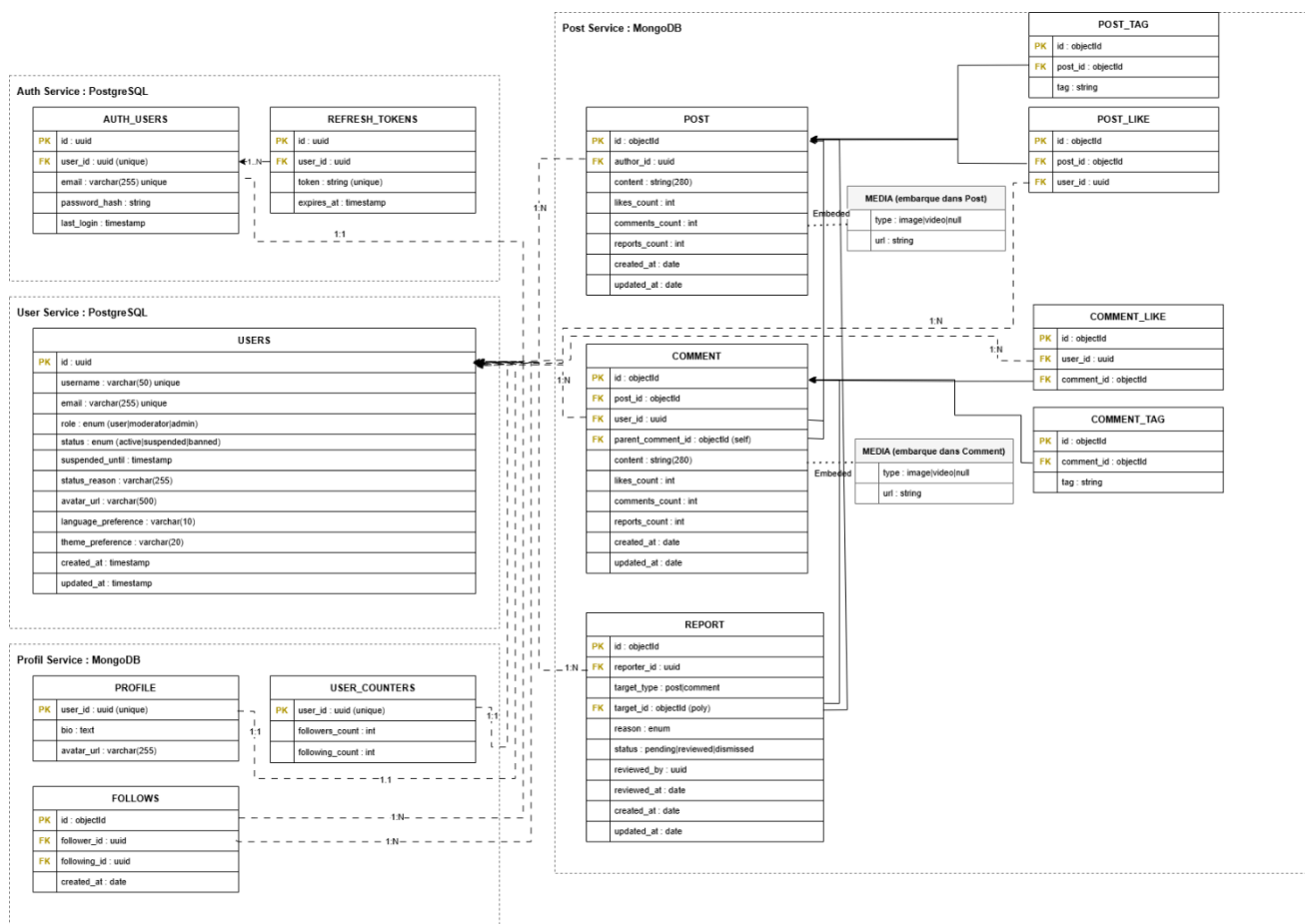
Architecture microservices : Breezy



Traits pleins : Routage via la passerelle et accès base dédiée.
Pointillés : Appels internes entre services (révocation).

4 Modèle de données

Le modèle de données découle directement des choix de stockage présentés en section 3.3 (database-per-service, PostgreSQL pour Auth et User, MongoDB pour Post et Profile). Cette section en détaille, service par service, les entités, leurs champs et leurs relations. La figure présente le modèle de données global, regroupé par microservice.



4.1 Service Auth (PostgreSQL)

Le service d'authentification isole les données sensibles liées à la connexion. La table AUTH_USERS stocke l'identifiant de l'utilisateur, son adresse e-mail unique et le condensat du mot de passe (password_hash), jamais le mot de passe en clair. Elle prend ainsi en charge la création de compte (Fx1) et l'authentification sécurisée (Fx2). La table REFRESH_TOKENS conserve les jetons de rafraîchissement avec leur date d'expiration. Cette séparation permet de révoquer ou de faire expirer une session sans toucher aux autres données du compte.

4.2 Service User (PostgreSQL)

La table USERS centralise les informations de compte et de présentation : username et email uniques, avatar_url, ainsi que les préférences language_preference (Fx22) et de theme_preference (Fx23). Deux champs énumérés y portent toute la logique d'autorisation : role (user | moderator | admin), qui matérialise la matrice des permissions, et status (active | suspended | banned), complété par suspended_until et status_reason, qui supporte la suspension et le bannissement des utilisateurs par la modération (Fx21). L'identifiant id de cette table est l'UUID pivot réutilisé par tous les autres services pour désigner un utilisateur.

4.3 Service Profile (MongoDB)

Le service de profil gère la présentation publique et le graphe social. La collection PROFILE contient la biographie et l'avatar de l'utilisateur (Fx10), reliés à USERS par le même user_id. La collection FOLLOWS enregistre chaque relation d'abonnement sous la forme d'un couple follower_id / following_id (Fx9). Pour éviter de recompter ces relations à chaque affichage, la collection USER_COUNTERS maintient des compteurs dénormalisés d'abonnés et d'abonnements, mis à jour de façon incrémentale lors de chaque abonnement ou désabonnement.

4.4 Service Post (MongoDB)

Cœur fonctionnel de l'application, ce service modélise les contenus et leurs interactions. La collection POST porte le message court (content, limité à 280 caractères, Fx3), son auteur (author_id), des compteurs dénormalisés (likes_count, comments_count, reports_count) et un sous-document MEDIA embarqué décrivant l'éventuelle image ou vidéo associée (Fx18, Fx19). Le champ auto-référencé parent_post permet de relier un message à un autre. La collection COMMENT suit la même structure et introduit un champ auto-référencé parent_comment_id, qui autorise des fils de discussion imbriqués : on répond ainsi aussi bien à un post (Fx7) qu'à un commentaire (Fx8).

Les interactions et la catégorisation sont déportées dans des collections dédiées. POST_LIKE et COMMENT_LIKE enregistrent les « j'aime » (Fx6), POST_TAG et COMMENT_TAG associent des tags aux contenus (Fx12) et alimentent, avec un index textuel sur le contenu, la recherche par mots-clés et par tags (Fx13). Enfin, la collection REPORT gère le signalement de contenu inapproprié (Fx20) : grâce à un couple target_type (post | comment) et target_id, un modèle polymorphe unique permet de signaler indifféremment un post ou un commentaire, tandis que les champs status, reviewed_by et reviewed_at tracent le traitement par la modération.

4.5 Principaux choix de conception

Plusieurs décisions structurantes méritent d'être soulignées :

- **Liens par UUID, sans clé étrangère inter-services.** Chaque entité référence un utilisateur par son UUID plutôt que par une clé étrangère, aucune contrainte ne pouvant traverser deux bases, la cohérence référentielle est donc assurée applicativement.
- **Compteurs dénormalisés.** Les champs `likes_count`, `comments_count`, `reports_count`, `followers_count` et `following_count` sont mis à jour à chaque action, ce qui rend l'affichage du fil et des profils très rapide en évitant des agrégations coûteuses, au prix d'une cohérence à maintenir applicativement.
- **Média embarqué + stockage objet.** Le sous-document MEDIA ne stocke que le type et l'URL, le fichier lui-même réside dans le stockage objet (MinIO/S3). La base reste légère et la lecture d'un post reste atomique.
- **Unicité garantie en base.** Des index composés uniques sur (`user_id`, `post_id`) pour les « j'aime » et (`follower_id`, `following_id`) pour les abonnements empêchent les doublons directement au niveau du stockage.
- **Auto-référencement pour les fils.** Les champs `parent_post` et `parent_comment_id` permettent une arborescence de réponses sans table de jointure supplémentaire.

5 Mise en œuvre des fonctionnalités

Cette section décrit la réalisation concrète des fonctionnalités obligatoires. Elle s'appuie sur l'architecture et le modèle de données présentés précédemment et détaille, pour les parcours les plus structurants, l'enchaînement des traitements entre la passerelle et les microservices.

5.1 Fonctionnalités principales

Le pipeline commun de traitement d'une requête

Toute requête suit le pipeline de la passerelle décrit en section 3.1

Création de compte avec validation (Fx1)

L'inscription est le parcours le plus orchestré, car il met en jeu trois services. Elle se déroule ainsi :

1. Le frontend envoie le nom d'utilisateur, l'e-mail et le mot de passe à l'auth-service (via la passerelle, route publique).
2. L'auth-service valide les données (format de l'e-mail, longueur du mot de passe, unicité) et hache le mot de passe avec bcrypt.
3. Il crée l'enregistrement dans AUTH_USERS (PostgreSQL) en générant un UUID qui servira d'identifiant pivot.
4. Avec ce même UUID, il appelle le user-service via l'api gateway pour créer la ligne USERS, puis le profile-service pour initialiser le PROFILE et les compteurs USER_COUNTERS.
5. En cas d'échec d'une de ces étapes, un mécanisme de rollback supprime les données déjà créées afin de ne laisser aucun compte incohérent réparti entre les bases.
6. Une fois le compte complet, l'auth-service génère les jetons et pose les cookies, connectant directement l'utilisateur.

Authentification sécurisée (Fx2)

À la connexion, l'auth-service compare le mot de passe fourni au condensat stocké (bcrypt), puis vérifie le statut du compte (un compte suspendu ou banni est refusé). Il émet alors deux jetons : un jeton d'accès de courte durée (15 minutes) et un jeton de rafraîchissement de longue durée (7 jours), ce dernier étant enregistré dans REFRESH_TOKENS. Les deux sont déposés dans des cookies HttpOnly, inaccessibles au JavaScript du navigateur, ce qui protège contre le vol de jeton par injection. Lorsque le jeton d'accès expire, le frontend obtient un nouveau jeton via la route de rafraîchissement sans redemander les identifiants, la déconnexion révoque le jeton de rafraîchissement.

Publication de messages courts, avec média et tags (Fx3)

La publication illustre la coopération entre le post-service et le media-service :

1. Si le message comporte une image ou une vidéo, le frontend l'envoie d'abord au media-service, qui valide le type et la taille du fichier (limite de 15 Mo), le pousse dans le bucket MinIO et renvoie une URL.

2. Le frontend appelle ensuite le post-service avec le contenu textuel (limité à 280 caractères), les tags éventuels et, le cas échéant, le média sous forme de sous-document { type, url } embarqué dans le post.
3. Le post-service identifie l'auteur via l'en-tête x-user-id de confiance, crée le document POST dans MongoDB et enregistre les tags associés (POST_TAG).

Le message est alors immédiatement disponible sur le profil de l'auteur et dans le fil de ses abonnés.

Fil d'actualité chronologique (Fx5)

Le fil agrège les publications des comptes suivis :

1. Le frontend récupère la liste des comptes suivis, puis interroge le post-service en lui transmettant ces identifiants.
2. Il interroge sa collection POST pour ne retenir que les messages dont l'auteur figure dans cette liste.
3. Les résultats sont triés par date décroissante (created_at), puis le frontend **enrichit** chaque post avec le nom et l'avatar de l'auteur en interrogeant le user-service à partir des UUID, puisqu'aucune jointure inter-bases n'est possible.

Profil et liste des messages publiés (Fx4, Fx10, Fx11)

La page de profil combine des données issues de deux bases : le user-service fournit le nom d'utilisateur, tandis que le profile-service fournit la biographie, l'avatar et les compteurs (abonnés, abonnements). Le post-service fournit les messages de l'utilisateur, filtrés selon l'onglet sélectionné, Posts, Posts et réponses ou J'aime, ce qui couvre l'affichage des messages sur le profil (Fx4) et la liste complète des publications (Fx11).

Interactions sociales : likes, réponses et abonnements (Fx6 à Fx9)

Ces fonctionnalités reposent sur des collections dédiées et des compteurs dénormalisés. Un like (Fx6) crée un document POST_LIKE (ou COMMENT_LIKE) protégé par un index unique sur le couple (user_id, post_id), qui empêche tout double like, et incrémente le compteur likes_count. Une réponse à un post (Fx7) crée un commentaire rattaché par post_id, une réponse à un commentaire (Fx8) utilise le champ auto-référencé parent_comment_id, ce qui permet des fils de discussion imbriqués. Un abonnement (Fx9) crée un document FOLLOWS (follower_id, following_id), lui aussi protégé par un index unique, et met à jour les compteurs d'abonnés et d'abonnements des deux utilisateurs concernés, le désabonnement effectue l'opération inverse.

5.2 Périmètre réalisé

Le tableau ci-dessous récapitule l'état de réalisation des 23 fonctionnalités du cahier des charges. L'intégralité des fonctionnalités obligatoires (Fx1 à Fx11) a été implémentée. Plusieurs fonctionnalités secondaires l'ont également été, au-delà du socle minimal.

Fx	Fonctionnalité	État
Fx1	Création de compte avec validation	Réalisé
Fx2	Authentification sécurisée (JWT)	Réalisé
Fx3	Publication de messages courts	Réalisé
Fx4	Affichage des messages sur le profil	Réalisé
Fx5	Fil d'actualité chronologique	Réalisé
Fx6	Liker un post	Réalisé
Fx7	Répondre à un post	Réalisé
Fx8	Répondre à un commentaire	Réalisé
Fx9	Suivre / être suivi	Réalisé
Fx10	Profil avec informations de base	Réalisé
Fx11	Liste des messages sur le profil	Réalisé
Fx12	Ajout de tags aux messages	Réalisé
Fx13	Recherche (posts, tags, utilisateurs)	Réalisé
Fx14	Notifications pour les mentions	Non réalisé
Fx15	Notifications pour les likes	Non réalisé
Fx16	Notifications pour les nouveaux abonnés	Non réalisé
Fx17	Messagerie privée	Non réalisé
Fx18	Ajout d'images aux messages	Réalisé
Fx19	Ajout de vidéos aux messages	Réalisé
Fx20	Signalement de contenu	Réalisé
Fx21	Suspension / bannissement	Réalisé

Fx22	Interface multilingue	Réalisé
Fx23	Thème personnalisable	Réalisé

6 Organisation, méthodologie et déroulement du projet

6.1 Gestion des rôles

Dans notre application, il est demandé de gérer plusieurs rôles : visiteurs, utilisateurs, modérateurs, et administrateur.

Afin de pouvoir plus facilement comprendre les différents accès aux fonctionnalités de l'application en fonction des différents rôles proposés, nous avons utilisé une matrice de permissions. Il s'agit de la première étape nous permettant de voir plus clair sur la répartition des tâches.

Fonctionnalité (Fx)	Visiteur	Utilisateur	Modérateur	Administrateur
Fx1. Création de comptes utilisateurs	✓	✗	✗	✓
Fx2. Authentification sécurisée	✗	✓	✓	✓
Fx3. Publication de messages courts	✗	✓	✓	✓
Fx4. Affichage des messages sur le profil	✗	✓ (seulement le sien)	✓	✓
Fx5. Flux chronologique des messages	✗	✓	✓	✓
Fx6. Liker un post	✗	✓	✓	✓
Fx7. Répondre à un post sous forme de commentaire	✗	✓	✓	✓
Fx8. Répondre à un commentaire sur un post	✗	✓	✓	✓
Fx9. Suivre ou être suivi par d'autres utilisateurs	✗	✓	✓	✓
Fx10. Profil utilisateur avec informations de base	✗	✓	✓	✓
Fx11. Liste des messages publiés par l'utilisateur sur le profil	✗	✓	✓	✓
Fx12. Ajout de tags aux messages	✗	✓	✓	✓
Fx13. Recherche de posts via des tags	✗	✓	✓	✓
Fx14. Notifications pour les mentions	✗	✓	✓ (modération)	✓ (administration)
Fx15. Notifications pour les likes	✗	✓	✗	✗
Fx16. Notifications pour les nouveaux followers	✗	✓	✗	✗
Fx17. Système de messages privés entre utilisateurs	✗	✓	✓	✓
Fx18. Ajout d'images aux messages	✗	✓	✓	✓
Fx19. Ajout de vidéos aux messages	✗	✓	✓	✓
Fx20. Signalement de contenu inapproprié	✗	✓	✓	✓
Fx21. Suspension ou bannissement des utilisateurs	✗	✗	✓	✓
Fx22. Interface multi-langues	✗	✓	✓	✓
Fx23. Thème personnalisé	✓	✓	✓	✓

6.2 Hiérarchie des tâches

L'ordre des tâches suit une logique de dépendances. Cette logique s'applique à deux échelles.

À l'échelle d'un service, le développement commence par les modèles de données. On définit d'abord les modèles, qui fixent la forme des données et servent de contrat à tout le reste, le premier travail métier du dépôt a d'ailleurs été la documentation du schéma de base de données.

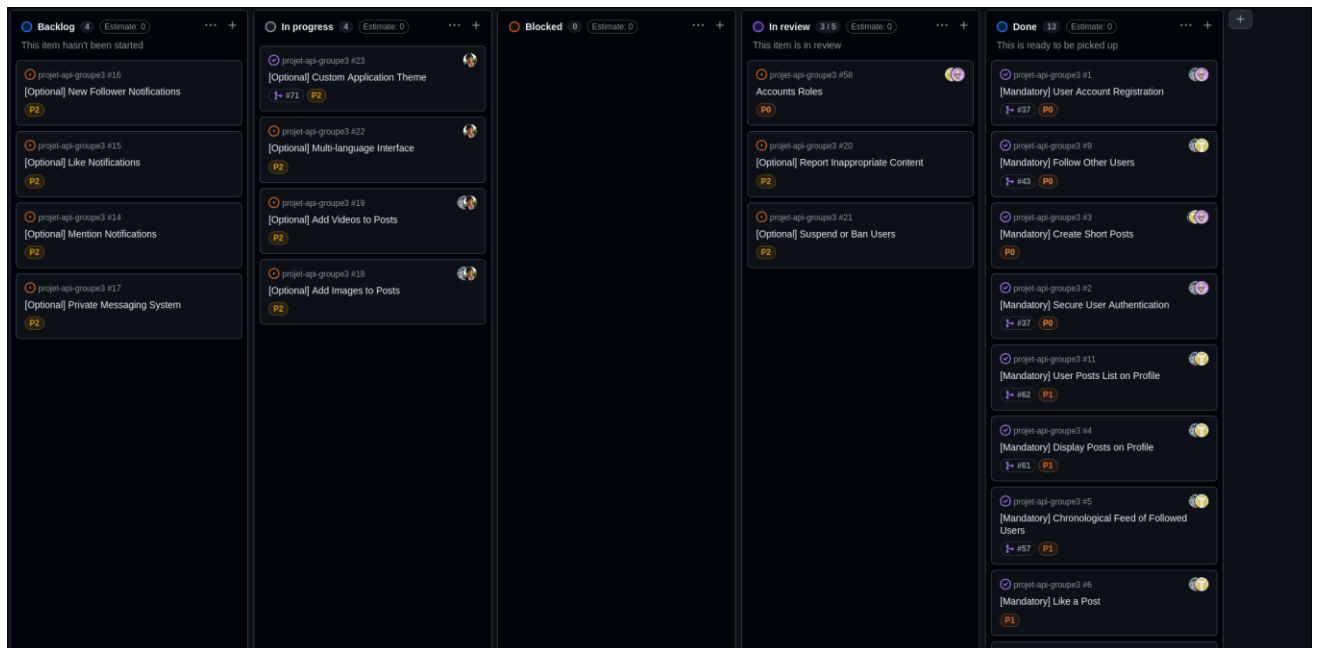
Viennent ensuite les controllers, qui portent la logique métier et manipulent ces modèles, puis les routes, qui associent un chemin HTTP à une fonction de controller déjà écrite, puis les middlewares transverses (authentification, validation, gestion des erreurs), qui enveloppent des routes existantes, la validation, en particulier, suppose de connaître les champs du modèle. Le seed vient en dernier, une fois un chemin d'écriture fonctionnel.

À l'échelle du projet, on développe d'abord les services (user, post), car l'utilisateur est référencé partout. Le profile-service (abonnements) suppose l'existence des utilisateurs, l'auth-service, qui orchestre la création du compte et du profil à l'inscription, a besoin que ces services existent déjà. La passerelle est construite ensuite, sa table de routes ne faisant que refléter les points d'entrée des services. Le frontend est intégré une fois le contrat d'API stabilisé. Les enrichissements (recherche, tags, médias, modération) s'ajoutent par-dessus un socle fonctionnel.

6.3 Méthodologie de travail

Le projet a été mené de façon incrémentale et collaborative, avec Git. Le code est regroupé dans un dépôt unique rassemblant le frontend et les services. Chaque fonctionnalité est développée sur une branche dédiée (feature/...) puis intégrée à main par une pull request, encadrée par un gabarit de PR. Une intégration continue (GitHub Actions) reconstruit les images Docker et vérifie le démarrage des conteneurs à chaque modification. Les différentes tâches sont suivies par des issues sur Github.

Nous avons aussi mis en place un kanban, permettant un suivi de l'état des tâches, de leurs affectations, etc...



Ici, pour chacune des fonctionnalités relevées, nous avons pu définir une priorité, qui était chargée de réaliser la tâche et dans quel état elle était :

- Backlog : Pour les tâches non commencées
- In progress : Pour les tâches en cours
- Blocked : Pour les tâches qui sont arrêtées suite à des difficultés de réalisation
- In review : Pour les tâches terminées qui attendent d'être validées
- Done : Pour les tâches terminées et validées

Cette organisation a été réellement bénéfique au projet, nous permettant d'en avoir une vision globale facilitant largement sa gestion.

La démarche est par ailleurs nettement itérative : l'historique montre plusieurs refactorisations d'amélioration (passage de JavaScript à TypeScript, migration du SQL brut vers l'ORM Prisma, réorganisation des dossiers), montrant une logique d'amélioration continue plutôt qu'un développement figé du premier coup.

6.4 Étapes du projet

Le déroulement, tel qu'on le lit dans l'historique du dépôt, s'organise en phases successives :

1. **Infrastructure et configuration** : Initialisation du dépôt, configuration de chaque service (paquets, TypeScript, Docker).
2. **Conception des données** : Documentation du schéma de base de données.
3. **Développement du cœur métier** : CRUD service par service (user, post, profil, puis auth avec les jetons JWT), suivi de la migration vers Prisma pour les services PostgreSQL.
4. **Passerelle d'API** : Mise en place de la gateway et du contrôle d'accès par rôles.
5. **Frontend** : Initialisation de l'interface, page d'inscription, internationalisation, puis intégration avec l'API d'authentification.
6. **Enrichissements** : Recherche (posts, commentaires, utilisateurs), tags, service de médias, page de profil et abonnements, modération.
7. **Automatisation** : Mise en place de la CI, des scripts de seed, et corrections de configuration Docker.

6.5 Ressources mobilisées

Ressources techniques :

- Backend : Node.js, TypeScript et Express pour tous les services, Prisma avec PostgreSQL (auth, user), Mongoose avec MongoDB (post, profile), Redis (révocation de jetons), jsonwebtoken et bcryptjs pour la sécurité, MinIO pour les médias, http-proxy-middleware pour la passerelle.
- Frontend : Next.js, React, TailwindCSS, Axios et i18next.
- Outils et infrastructure : Docker et Docker Compose pour l'orchestration, GitHub et GitHub Actions pour le versionnage et l'intégration continue, draw.io pour les schémas d'architecture.

7 Axes d'amélioration et évolutions potentielles

Le socle fonctionnel étant en place, plusieurs évolutions permettraient de compléter le périmètre et d'industrialiser l'application.

Compléter le périmètre fonctionnel. Les fonctionnalités secondaires non traitées sont les candidates naturelles. Un système de notifications (Fx14 à Fx16) reposerait sur un service dédié et une communication temps réel (WebSocket ou Server-Sent Events) afin d'avertir l'utilisateur d'une mention, d'un like ou d'un nouvel abonné. Une messagerie privée (Fx17) nécessiterait un service de conversations avec sa propre base et, idéalement, le même canal temps réel.

Renforcer la qualité et la fiabilité. L'ajout d'une suite de tests automatisés (tests unitaires des controllers, tests d'intégration des routes) exécutée dans la CI sécuriserait les évolutions futures et constituerait l'amélioration la plus structurante à court terme. Elle pourrait être complétée par une couche d'observabilité (journalisation centralisée, métriques), les health checks étant déjà en place.

Durcir la sécurité et préparer la production. La mise en place d'un rate-limiting sur la passerelle limiterait les abus (force brute, spam). Ensuite, le déploiement en production (un fichier de configuration cloud a déjà été préparé) finaliserait la chaîne : HTTPS, gestion des secrets et bases managées. Enfin, l'ajout d'un répartiteur de charge (load balancer) devant la passerelle, associé à plusieurs instances des services, permettrait d'absorber la montée en charge.

Affiner l'expérience utilisateur au fil des retours. Une fois l'application mise entre les mains de premiers utilisateurs, leurs retours constitueraient une base concrète pour prioriser les améliorations d'ergonomie et d'interface : clarté des parcours, lisibilité du fil, accessibilité, cohérence visuelle entre les écrans. Cette démarche itérative, fondée sur l'usage réel plutôt que sur des suppositions, permettrait d'ajuster l'UX/UI au plus près des attentes et d'augmenter l'adoption.

8 Conclusion

Le projet Breezy a abouti à un réseau social fonctionnel, conforme aux objectifs fixés. L'intégralité des fonctionnalités obligatoires (Fx1 à Fx11) a été réalisée, complétée par plusieurs fonctionnalités secondaires : tags et recherche, médias, signalement, modération, interface multilingue et thème personnalisable. Sur le plan technique, les contraintes ont été respectées : une architecture microservices organisée autour d'une passerelle d'API unique, des bases isolées selon le patron database-per-service (PostgreSQL, MongoDB et stockage objet MinIO), le tout conteneurisé avec Docker et appuyé par une intégration continue.

Au-delà du résultat, ce projet a été une mise en pratique concrète d'une architecture distribuée et du travail collaboratif, structuré par Git et un suivi kanban. Le socle étant solide et extensible, les fonctionnalités restantes (notifications, messagerie privée) et les chantiers d'industrialisation (tests, sécurité, mise en production) pourront s'y greffer sans remettre en cause les choix de conception.

Enfin, la documentation de l'API et les schémas en bonne résolution (.png) sont disponibles dans le dossier doc du dépôt GitHub.